

Use Wave File Bypass CSP

CSP Introduction

CSP全称Content Security Policy，即内容安全策略。CSP是一个额外的安全层，用于检测并削弱某些特定类型的攻击，包括XSS和注入。

CSP被设计为完全向后兼容，在不同的浏览器上，不会因是否支持CSP而产生冲突问题。在不进行特定设置之前，默认为网页使用标准的同源策略。

通常可以使用meta标签来设置CSP，或者使用php的header函数来进行相关属性的设置。

CSP的配置会直接影响到页面加载资源的方式，在适当配置的情况下，可以有效的防范XSS攻击。

CSP Configuration

我们通过Content-Security-Policy头信息来进行CSP的设置，通常格式如下：

Content-Security-Policy: policy

其中，policy部分对应的为具体的内容安全策略。

一个策略由一系列的策略指令组成，每个策略都描述了一个针对某个特定类型资源以及生效范围的策略。

网上对于相关指令和资源表的说明已经很多了，我就不再赘述。

Bypass CSP With Different Policy

Common Policy Bypass

目前在比赛中常见的绕过CSP一般是：

```
script-src 'self' 'unsafe-inline'
script-src 'self' 'unsafe-eval'
script-src 'nonce-*****'
```

通常情况下，除了CSP之外，还都会搭配一定的过滤措施来让选手进行绕过。

这里有几个例子，我就不再多说：

1. oCTF 2018 h4x0rs.club2 writeup

<http://sec2hack.com/ctf/octf2018-h4x0rs-club2.html>

2. Google CTF 2016 Wallowing Wallabies - Part Three

<http://countersite.org/articles/web-vulnerability/83-web-writeup-googlectf-wallowing-wallabies.html>

Bypass "script-src 'self' "

在前几天的PlaidCTF中，出现了如下的CSP：

```
Content-Security-Policy: style-src 'self' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com; media-src 'self' blob:; script-src 'self'; object-src 'self'; frame-src 'self'
```

script-src 'self'代表着只能加载符合同源策略的文件，直接插入至html页面中的静态script标签将无法执行。结合其他CSP来看，常用的iframe，object等标签也是无法被绕过的。

我尝试着使用link的预加载机制去带出cookie，然而受限于script-src 'self'的限制，虽然能够通过dns带出信息，但是无法将cookie带出来，因此预加载也是无法使用的。

于是只能另外寻找突破口，在查阅大量资料后发现，可以通过引入正常的非js文件来达到引入js脚本的效果。考虑到题目中具有上传点，可以将js代码插入到尾部来进行绕过。

本地搭建环境进行测试：

xss.html内容为：



```
1 <script src="./cc.wave"></script>
```

cc.wave内容为:

test.phpcc.wave✕xss.htmlaaa.php

1aaaaaaaaaaaaaa/*bbbbbbbbbbbbbbbbbb*/="test";alert(1);|

执行效果为:



可以看到注释部分并未对js的执行起到干扰，因此这种攻击手法是可行的。

在进行上传时，后端会进行文件格式校验，因此需要在保证文件格式验证正确的情况进行绕过，在录音选项中，上传的文件为webm格式，文件头是不可见字符，在引入js文件时，会产生错误，因此需要引入文件的文件头是可见字符。

这里对比下两者的文件格式便很明显了:

77	05	75	70	01	75	05	77	05	01	00	5a	20	00	01	72	Disposition: fo
6d	2d	64	61	74	61	3b	20	6e	61	6d	65	3d	22	61	75	m-data; name="au
64	69	6f	66	69	6c	65	22	3b	20	66	69	6c	65	6e	61	diofile"; filena
6d	65	3d	22	22	0d	0a	43	6f	6e	74	65	6e	74	2d	54	me="" Content-T
79	70	65	3a	20	61	70	70	6c	69	63	61	74	69	6f	6e	ype: application
2f	6f	63	74	65	74	2d	73	74	72	65	61	6d	0d	0a	0d	/octet-stream
0a	0d	0a	2d	2d	2d	2d	2d	2d	57	65	62	4b	69	74	46	-----WebKitF
6f	72	6d	42	6f	75	6e	64	61	72	79	59	4a	41	71	59	ormBoundaryYJAqY
4c	68	54	72	6b	31	4e	59	78	46	4d	0d	0a	43	6f	6e	LhTrk1NYxFM Con
74	65	6e	74	2d	44	69	73	70	6f	73	69	74	69	6f	6e	tent-Disposition
3a	20	66	6f	72	6d	2d	64	61	74	61	3b	20	6e	61	6d	: form-data; nam
65	3d	22	61	75	64	69	6f	66	69	6c	65	22	3b	20	66	e="audiofile"; f
69	6c	65	6e	61	6d	65	3d	22	61	75	64	69	6f	2e	77	ilename="audio.w
65	62	6d	22	0d	0a	43	6f	6e	74	65	6e	74	2d	54	79	ebm" Content-Ty
70	65	3a	20	61	70	70	6c	69	63	61	74	69	6f	6e	2f	pe: application/
6f	63	74	65	74	2d	73	74	72	65	61	6d	0d	0a	0d	0a	octet-stream
1a	45	df	a3	9f	42	86	81	01	42	f7	81	01	42	f2	81	EËf B ·B÷·Bò·
04	42	f3	81	08	42	82	84	77	65	62	6d	42	87	81	04	Bó·B-·webmB ·
42	85	81	02	18	53	80	67	01	ff	ff	ff	ff	ff	ff	ff	B · Sëgyyyyyyy
15	49	a9	66	99	2a	d7	b1	83	0f	42	40	4d	80	86	43	l0f *x± B@M€ C
68	72	6f	6d	65	57	41	86	43	68	72	6f	6d	65	16	54	hromeWA Chrome T
ae	6b	bf	ae	bd	d7	81	01	73	c5	87	7c	f6	c1	7b	ae	®k¿®½x·sÅ ôÁ{®
8f	b5	83	81	02	86	86	41	5f	4f	50	55	53	63	a2	93	μ · A_OPUSç¢
4f	70	75	73	48	65	61	64	01	01	00	00	80	bb	00	00	OpusHead€»
00	00	00	e1	8d	b5	84	47	3b	80	00	9f	81	01	62	64	á μ_G;€ ·bd
81	10	1f	43	b6	75	01	ff	ff	ff	ff	ff	ff	ff	e7	81	· C¶uyyyyyyyç·
00	a3	41	7b	81	00	00	80	fb	83	02	d1	ff	fe	8a	a2	fA{·eû Ñÿb ¢
af	40	ed	6b	31	a0	d0	30	d9	93	8a	f3	af	72	8e	0a	~@ík1 Ð0Ù ó`r
9f	48	e2	5c	b7	a6	ca	f0	86	bc	b5	e9	29	3a	38	ee	Hâ\·îËð ¼µé):8î
f2	49	10	fd	3a	15	da	19	3f	25	86	34	cd	4f	0a	6f	ðl ú: ú 7% 4Í0 o

29	73	70	6f	73	69	74	69	6f	6e	3a	20	66	6f	72	6d	2d	sposition: form-
2a	64	61	74	61	3b	20	6e	61	6d	65	3d	22	64	65	73	63	data; name="desc
2b	72	69	70	74	69	6f	6e	22	0d	0a	0d	0a	62	0d	0a	2d	ription" b -
2c	2d	2d	2d	2d	2d	57	65	62	4b	69	74	46	6f	72	6d	42	-----WebKitFormB
2d	6f	75	6e	64	61	72	79	5a	65	4b	72	68	73	49	52	57	oundaryZeKrhslRW
2e	59	4d	67	35	53	6c	50	0d	0a	43	6f	6e	74	65	6e	74	YMg5SLIP Content
2f	2d	44	69	73	70	6f	73	69	74	69	6f	6e	3a	20	66	6f	-Disposition: fo
30	72	6d	2d	64	61	74	61	3b	20	6e	61	6d	65	3d	22	61	rm-data; name="a
31	75	64	69	6f	66	69	6c	65	22	3b	20	66	69	6c	65	6e	udiofile"; file
32	61	6d	65	3d	22	e5	bd	95	e9	9f	b3	2d	30	30	32	2e	ame="ã½ é ³-002.
33	77	61	76	22	0d	0a	43	6f	6e	74	65	6e	74	2d	54	79	wav" Content-Ty
34	70	65	3a	20	61	75	64	69	6f	2f	77	61	76	0d	0a	0d	pe: audio/wav
35	0a	52	49	46	46	a4	9c	01	00	57	41	56	45	66	6d	74	RIFFα WAVEfmt
36	20	10	00	00	00	01	00	01	00	80	bb	00	00	00	77	01	€»w
37	00	02	00	10	00	64	61	74	61	80	9c	01	00	00	00	00	data€
38	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
39	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
3a	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
3b	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

wav格式的文件是以RIFF明文开头的，可以使用我上面所用到的攻击方法去构造xss代码，而webm开头为乱码，在执行时，会因为产生报错而中止执行。

在绕过文件格式检查之后，js会根据文件格式给定一个MIME-TYPE，在带入src属性的时候，audio的Type会和可执行脚本产生冲突，因此wav文件无法代入，而wave在MIME转换的名单之外，因此在上传成功wave文件时，其MIME-TYPE并不会与src冲突。

因此直接在description中写入：

即可拿到cookie作为idolt1登陆，接下来的就是审计js，上传wave文件动态添加xss代码即可。

Bypass文件格式的重点在于javascript在遇到”变量+运算符+变量”格式的表达式时，可以将注释插入其中，并且不会产生干扰。

在这个题目中，难点有两个：

1. 绕过self限制
2. 构造出符合文件内容检查的文件

同样的，我对于php是否会有此类特性也感到好奇，在经过测试后发现，在插入注释后，php的执行也不会被干扰。而python因为不具有行内注释的操作而无法做到上述操作。

之前看某篇paper，其中曾提到使用GIF绕过CSP，其中的部分思路和我上面用到的有部分相似。Paper链接如下：

https://www.slideshare.net/xoomario/jsmvcomfg-to-sternly-look-at-javascript-mvc-and-templating-frameworks/3-TodayJavaScript_MVC_Templating_FrameworksWhy_Because

首先，同样是上传GIF，使得GIF与目标网站处于同源下，然后使用Angular的class去调用这个GIF文件，然后会生成script标签，并且其src属性是GIF文件，此时GIF中的内容被作为js代码解析了，并且因为处于同源情况下，因此可以顺利触发xss。

Comments On CSP

CSP作为内容安全策略，在合理配置的情况，可以极大的提高xss的攻击成本，以达到较好的防御效果，然而部署成本同样较高，一是熟练掌握相关策略带来的难度，一是配置CSP所带来的工作量。

CSP的不当配置不仅会引发安全问题，还有可能导致页面资源加载失败，但总的来说，CSP仍然是防范XSS攻击较为优秀的措施。